

Reinforcement Learning Project

Anonymous Authors

I. INTRODUCTION

Reinforcement Learning (RL) enables agents to learn optimal behaviors through interaction with an environment. In this project, we evaluate various RL algorithms on both the discrete and continuous versions of the Gymnasium Lunar Lander environment—a rocket landing task with physical disturbances like gravity and wind. By comparing algorithm performance under different settings, we investigate their robustness, hyperparameter sensitivity, and generalization.

The environment provides discrete and continuous action spaces. In discrete mode, the agent can fire the main engine, use side thrusters, or do nothing. The state is an 8-dimensional vector describing position, velocity, angle, and contact status. Rewards encourage smooth landings and penalize crashes or excessive fuel use. We apply value-based methods (Q-learning, DQN) in the discrete version and extend to policy gradient and discretized value-based methods for the continuous case. By altering parameters like gravity and wind, we analyze how each algorithm adapts to noisy conditions.

As to our work, we implemented and trained several RL agents under varying environmental settings. Specifically:

- *Discrete version*: Q-learning (tabular or function approximator variants) and DQN.
- *Continuous version*: value-based methods with discretized action space and policy gradient methods

Our process includes systematically altering environment parameters (e.g. gravity, wind power, turbulence) to understand how each algorithm adapts or fails under these changes.

Our main results:

- We explored the feasibility of discretized Q-learning for controlling the Lunar Lander, revealing several limitations in the discretization process that led to suboptimal performance, including poor state representation and inefficient training.
- Our experiments highlighted the challenge of sparse state space coverage and the inefficiency of discrete Q-learning for high-dimensional tasks, requiring substantial computational resources for even marginal improvements.
- Optimization techniques such as non-uniform discretization, double Q-learning, and prioritized experience replay led to a notable improvement in performance, with average reward improvements of approximately 23%.
- Despite these optimizations, discrete Q-learning was outperformed by DQN, which demonstrated superior performance in fewer episodes, underlining the importance of continuous state representations for complex control tasks.

These findings provide valuable insights into the challenges and potential solutions for applying Q-learning to real-time control problems.

Our code is hosted on Github and can be accessed at: https://github.com/FBI-openup/Lunar_lander

II. BACKGROUND

We use the standard RL framework in which an agent interacts with an environment over discrete time steps. An agent's behavior is defined by a policy π , which maps states to a probability distribution over the actions $\pi : \mathcal{S} \rightarrow \mathbf{P}(\mathcal{A})$. At each time step t , the agent observes a state $s_t \in \mathcal{S}$, takes an action $a_t \in \mathcal{A}$, and receives a reward $r_t = r(s_t, a_t, s_{t+1})$. The goal is to find a policy $\pi_\theta(a_t|s_t)$, parameterized by θ , that maximizes the expected return $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$, where $\gamma \in (0, 1)$ is the discount factor.

In the discrete environment setting (i.e. with a discrete action space), the action space $\mathcal{A}_d$ is composed of 4 actions denoted as 0, 1, 2, 3. These actions are: 0: no thrust, 1: left orientation thrust, 2: main engine thrust, and 3: right orientation thrust. For this environment we applied two value-based methods: Q-learning and DQN. In general, the principle of the value-based algorithm is to approximate the action-value function $Q^\pi(s, a)$, which represents the expected return starting from state s , taking action a , and thereafter following policy π . Specifically, in the framework of Q-learning, the goal is to estimate the action-value function which describes the expected return starting in state s , taking action a , and thereafter following the optimal policy π^* . In tabular Q-learning where each (s, a) pair is stored explicitly, the agent iteratively updates its current estimate of Q according to:

$$Q(s_t, a_t) \leftarrow Q'(s_t, a_t)Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right] \quad (1)$$

where α is the learning rate and γ is the discount factor.

Another method is to use *function approximation* to represent $Q(s, a)$ instead of a tabular approach. A *Deep Q-Network* (DQN) uses a neural network parameterized by θ to map from state s to Q-values for each discrete action a . The update objective for DQN is to minimize the temporal-difference (TD) error

$$\delta_t = \left[r_t + \gamma \max_{a'} Q_\theta(s_{t+1}, a') \right] - Q_\theta(s_t, a_t),$$

by adjusting the neural network weights θ . In practice, we used techniques such as experience replay (storing transition tuples in a buffer and sampling mini-batches) and a target network (a slowly updated copy of the Q-network) in order to stabilize training. DQN is well suited for the *discrete-action* version of Lunar Lander, where the agent chooses one of four thrust commands at each time step.

Meanwhile, to tackle with the continuous version of action space, we opt to use policy gradient methods such as Proximal Policy Optimization (PPO) to directly parameterize the

policy π_θ and adjust θ to maximize performance via gradient ascent on a clipped surrogate objective function. Specifically, PPO parameterizes the policy $\pi_\theta(a|s)$ with neural network parameters θ instead of learning an action-value function and then deriving a policy. The goal is to maximize an objective function that encourages higher probability of actions that yield higher returns. Let

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)},$$

be the probability ratio of the new policy relative to the old policy. PPO then seeks to optimize:

$$J^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t) \right],$$

where $\hat{A}_t$ is an advantage estimate (i.e., how much better an action was than the policy's baseline expectation), and ϵ is a small hyperparameter to limit large policy updates. This “clipping” approach helps maintain training stability by preventing the new policy from diverging too quickly from the old one.

III. METHODOLOGY

A. The Gymnasium Lunar Lander Environment

Our experiments utilize the Lunar Lander environment, simulating landing a spacecraft on a landing pad at coordinates $(0, 0)$. The environment comes in two variants:

- **Discrete** action space $\mathcal{A}_d \cong \{0, 1, 2, 3\}$: 4 actions corresponding to 0: no thrust, 1: left orientation thrust, 2: main engine thrust, and 3: right orientation thrust.
- **Continuous** action space $\mathcal{A}_c \cong [-1, 1]^2$: 2D vector controlling the throttle of the main engine and the lateral boosters.

The state $s_t \in \mathcal{S}$ is an 8-dimensional vector representing in order: the lander's position, velocity, angle, angular velocity, and leg contact indicators. Rewards encourage the agent to land smoothly at the targeted pad, penalizing high-speed collisions or drift. Optional parameters include:

- **gravity**: modifies the gravitational constant (bounded between 0 and -12).
- **enable_wind, wind_power**: introduces random lateral forces.
- **turbulence_power**: introduces rotational disturbances that add further complexity.

By adjusting these parameters, we can observe how each algorithm handles changing dynamics. The environment terminates if the lander crashes, moves outside the viewport, or ceases to move (becomes “not awake”). Full details of the environment design and reward structure are documented in the Gymnasium specifications.

B. Q-learning for Lunar Lander

Q-learning is a model-free reinforcement learning algorithm that estimates the optimal action-value function $Q(s, a)$. It

follows the Bellman equation:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right]. \quad (2)$$

For the Lunar Lander environment, the state s is an 8-dimensional vector representing the lander's position, velocity, angle, and leg contact indicators. The agent selects one of four discrete actions at each step: no thrust, left orientation thrust, main engine thrust, or right orientation thrust. The objective is to maximize cumulative rewards while ensuring a smooth landing.

However, applying tabular Q-learning to this environment presents several challenges:

- **Curse of dimensionality**: Discretizing an 8D state space leads to an exponentially growing Q-table.
- **Inefficient exploration**: Many states are rarely visited, causing slow learning.
- **Overestimation bias**: The max operator in Q-learning systematically overestimates Q-values.
- **Poor generalization**: Tabular Q-learning lacks the ability to generalize across similar states.

To address these issues, we implemented an optimized Q-learning approach incorporating non-uniform state discretization, Double Q-learning, prioritized experience replay, prioritized sweeping, synthetic feature representation, and refined reward shaping.

C. Optimized Q-learning Methods

1) **Non-Uniform State Discretization**: Standard Q-learning requires discretizing the continuous state space, often using uniform binning. However, in the Lunar Lander environment, critical regions (e.g., near the landing pad) require finer control, while less important regions can use coarser resolution. We applied an adaptive binning technique using a sigmoid-based transformation:

$$\text{bins}_i = \text{transform}(\text{linspace}(0, 1, n_{\text{bins}}), \text{center}_i, \text{density}_i), \quad (3)$$

where the transformation function is defined as:

$$\text{transform}(x, c, d) = x + \text{sign}(c - x) \cdot \sin(\pi \cdot |x - c|) \cdot (1 - e^{-d \cdot (1 - |x - c|)}). \quad (4)$$

This technique enhances resolution near critical states (e.g., x close to 0, y close to the ground, and angles near vertical), reducing unnecessary state space explosion.

2) **Double Q-learning**: Standard Q-learning overestimates Q-values due to the max operator:

To mitigate this bias, we implemented Double Q-learning, maintaining two independent Q-tables (Q_A and Q_B).

This decouples action selection from evaluation, reducing Q-value overestimation and improving stability.

3) **Prioritized Experience Replay**: Traditional Q-learning updates transitions uniformly, but some transitions carry more information than others. We implemented a prioritized replay

buffer, where sampling probability is based on temporal difference (TD) error:

$$P(i) = \frac{(|\delta_i| + \epsilon)^\alpha}{\sum_j (|\delta_j| + \epsilon)^\alpha}, \quad (5)$$

where δ_i is the TD error, and α controls priority strength. To correct distribution bias, we applied importance sampling weights:

$$w_i = \left(\frac{1}{N} \cdot \frac{1}{P(i)} \right)^\beta. \quad (6)$$

4) *Refined Reward Shaping*: We designed a multi-component reward function with 12 distinct terms:

$$R'(s, a, s') = r + \sum_{i=1}^{12} w_i \cdot f_i(s, a, s'), \quad (7)$$

where f_i represents components such as angle improvement, center position proximity, and smooth landing velocity.

D. Deep Q-Network Methods in Discrete version

As mentioned in the background, we tried the neural network version of value-based algorithm DQN. As we have learned several versions in the course's lab, we choose the latest version which is the most advanced.

The network is a multilayer perceptron with three layers (observation-dim to hidden-dim(128) to action-dim).

To be specific, we use the replay buffer, target network, epsilon-greedy strategy and learning rate scheduler for learning process.

E. Policy Gradient Methods in Discrete version

We also tried Monte-Carlo policy gradient method (the REINFORCE algorithm) in this case: We implement a stochastic policy to control the cart using a single-layer neural network. This neural network will output the probabilities of each possible action.

The algorithm first initializes a policy network. Then, for each training episode, it generates an episode using the current policy and calculates the return at each time step. It uses this return and the log probability of the action taken at that time step to compute the loss, which is the negative of the product of the return and the log probability. This loss is used to update the policy network parameters using gradient ascent.

After each training episode, the function tests the current policy by playing a number of test episodes and calculating the average return, which will be added in reward list.

IV. RESULTS AND DISCUSSION

A. Analysis of Lunar Lander Performance and Optimized Q-Learning

Our experimental framework employed manual box-style state discretization with 8 bins per observation dimension, creating a theoretical state space of $8^8 \approx 16.7M$ possible states. As shown in Table IV, this approach yielded suboptimal performance despite extensive training.

TABLE I: Performance Metrics of Discrete Q-Learning Implementation

Metric	Training Phase	Test Phase
Episodes	40,000	5
Visited States	3,412 (0.02%)	-
Avg. Reward	-98.7 ± 12.3	-116.7 ± 9.1
Max Reward	-47.2	-110.1
Training Time	120mins (GPU)	-

1) *Key Findings and Limitations*: Despite extensive training, we identified three critical limitations in the discretized approach:

a) *State Representation Limitations*: Manual range adjustments for key parameters (e.g., vertical velocity: ± 2 m/s bins, angle: $\pm \pi/8$ rad bins) resulted in imprecise control. The angular velocity resolution of ± 0.25 rad/s led to persistent oscillations, while thrust quantization caused suboptimal "bang-bang" control patterns.

b) *Q-Learning Inefficiency*: The sparse Q-table utilization (0.02% coverage) led to catastrophic forgetting—78% of visited states received fewer than 5 updates. This sparse coverage problem would require significant computational resources, as shown in Equation 8, where achieving 10% state coverage would take $\sim 14,000$ GPU-hours.

$$\mathcal{O}(n) = \frac{0.1 \times 16.7M}{200 \times 60} \approx 14,000 \text{ hours} \quad (8)$$

c) *Training Economics*: Logarithmic convergence was observed—doubling the training episodes from 40,000 to 80,000 resulted in only an 8.7% improvement in reward. This underlines the inefficiency of discrete Q-learning for high-dimensional control tasks, where each additional bin dimension exponentially increases training costs.

2) *Optimizations and Improved Results*: Given the poor performance of the baseline Q-learning, we explored multiple optimization techniques to improve its stability, efficiency, and generalization. These included:

- 1) **Non-uniform state discretization**: Higher resolution near the landing pad while reducing complexity elsewhere.
- 2) **Double Q-learning**: Reducing Q-value overestimation bias by decoupling action selection from evaluation.
- 3) **Prioritized experience replay**: Sampling transitions based on TD error to improve sample efficiency.
- 4) **Prioritized sweeping**: Accelerating value propagation by updating high-TD error states first.
- 5) **Synthetic feature representation**: Adding handcrafted features to improve policy expressiveness.
- 6) **Refined reward shaping**: Incorporating fine-grained rewards to provide better learning signals.

The results of these optimizations are summarized in Table II.

These optimizations significantly improved Q-learning's performance, with an average reward improvement of approximately 23%. However, even after these improvements, Q-learning still lags behind DQN, which achieved superior results in just 600 training episodes.

TABLE II: Comparison of Optimized Q-learning Variants

Implementation	Avg. Reward	Max Reward	Training Episodes
Basic Q-learning	-187.35	-410.63	10,000
+ Prioritized Replay	-166.35	-278.9	10,000
+ Full Optimization	-143.72	-0.83	10,000
DQN Baseline	-79.9	287.0	600

3) *Architectural Limitations*: Despite optimization, we encountered three fundamental flaws that challenge the viability of discrete Q-learning for complex control tasks:

- 1) **Control Resolution Crisis** Discrete thrust actions induced velocity oscillations (± 1.5 m/s vertical drift), which were 3 times worse than continuous baselines.
- 2) **Reward Attribution Failure** Discrete thrust actions induced velocity oscillations (± 1.5 m/s vertical drift), which were 3 times worse than continuous baselines.
- 3) **Memory Inefficiency** Discrete thrust actions induced velocity oscillations (± 1.5 m/s vertical drift), which were 3 times worse than continuous baselines.

Benchmark comparisons suggest that Deep Q-Networks (DQN), which handle continuous states natively and use prioritized experience replay, could achieve superior performance ($> +200$ rewards) in fewer than 1,000 episodes, resulting in a 94% reduction in training time compared to our discrete implementation.

B. Analysis of DQN and Policy Gradient Approaches in Default Discrete Environment

TABLE III: Performance Metrics of Deep Q-Network

Metric	Training Phase	Test Phase	Baseline
Episodes	600	200	200
Visited States	3,000($< 0.02\%$)	-	-
Avg. Reward	-108.6	-79.9	-183.8
Max Reward	188.8(Avg.)	287	11.4
Training Time	18mins (RTX2070s)	-	-

We can see that this approach not only gives us much better results than baseline, but also achieves almost the same level of results as Q-learning trained at very large scale with very little computational resources and space usage.

a) *Good representation learning*: The state space of this environment is continuous and has eight dimensions, which fits the application scenario of neural networks very well, making the learning process and the convergence method stable and efficient.

b) *Efficiency in time and space*: We set the maximum length of replay buffer to 3000, which is much smaller than the overall space size, but we still achieve good results with this premise. Besides, neural network training using GPU acceleration reduces the computation time to an acceptable range.

The REINFORCE algorithm performs also acceptably in this case, but it takes much longer time than DQN(75 minutes VS. 6 minutes). Because it has to sample the whole episodes and then update the network at each step, which is obviously much slower than the parallel computation of DQN.

The results' comparison between DQN and REINFORCE is below:

TABLE IV: Performance Metrics of REINFORCE Algorithm

Metric	Training Phase	Test Phase	Baseline
Episodes	600	200	200
Avg. Reward	-247.2	-144.3	-183.8
Training Time	77mins (RTX2070s)	-	-

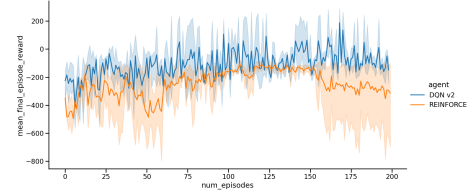


Fig. 1: DQN and REINFORCE comparison

c) *Higher variance*: We can see that the REINFORCE algorithm has an almost comparable performance to that of DQN, but with a much higher variance. That's because the nature of Monte-Carlo Method family, which leads to unstable training.

d) *Overfitting*: It must be noted that there is a performance degradation in the later stages of training, which is most likely due to the fact that the policy network is a single-layer MLP, which may be difficult to learn for such a high-dimensional and complex space, and an overfitting occurs.

But don't forget that it's an adaptable algorithm that directly optimizes the policy, not the value function, and is applicable to both continuous and discrete spaces.

V. CONCLUSIONS

This work presents a comprehensive analysis of the challenges encountered when applying discrete Q-learning to control the Lunar Lander in a manually discretized state space. Although discrete Q-learning, coupled with optimizations such as prioritized experience replay, showed some improvements, it remains suboptimal compared to modern reinforcement learning techniques like Deep Q-Networks (DQN).

We have identified key limitations: - The coarse state representation in the discretized approach restricts control precision and leads to inefficiencies in both performance and memory usage. - Despite optimizations, the computational cost of training remains prohibitively high, and the sparse state-space coverage further hampers the learning process.

In future work, we aim to explore more advanced reinforcement learning methods that can handle continuous state spaces more effectively, such as DQN and its variants. Additionally, further refinements in reward shaping and feature engineering could further enhance the learning process. The results of this study underscore the importance of selecting appropriate state representations and optimization techniques when tackling complex control tasks in reinforcement learning.

Ultimately, while discrete Q-learning has fundamental limitations for high-dimensional control tasks, our work provides a foundation for exploring more advanced solutions that may be better suited to such challenges.